Лабораторная работа 8 «Оптимизация архитектуры сетей глубокого обучения»

В этой лабораторной работы будут рассмотрены следующие вопросы:

- основные понятия искусственных нейронных сетей и глубокого обучения;
- набор данных Iris и классификатор на основе многослойного перцептрона (МСП);
- повышение качества классификатора посредством оптимизации архитектуры сети;
- дальнейшее повышение качества классификатора посредством объединения оптимизации архитектуры сети с настройкой гиперпараметров.

В этой лабораторной работе нам понадобится Python 3 и следующие библиотеки:

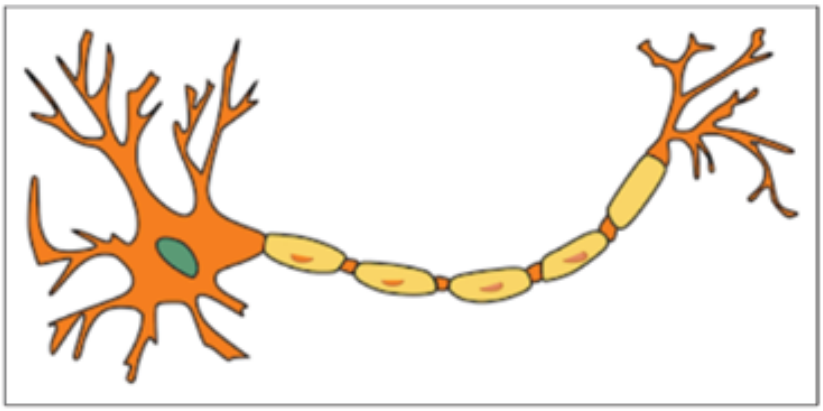
- deap;
- numpy;
- sklearn.

Кроме того, мы будем использовать набор данных Iris Калифорнийского университета в Ирвайне (https://archive.ics.uci.edu/ml/datasets/Iris).

Код приведенных в этой главе программ можно найти в репозитории книги на GitHub по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/tree/master/Chapter09. Чтобы увидеть код в действии, посмотрите видео по адресу http://bit.ly/317KCXA.

Искусственные нейронные сети и глубокое обучение

Нейронные сети – одна из самых широко используемых моделей в машинном обучении. В их основе лежат результаты исследований человеческого мозга. Главными структурными элементами являются нейроны, моделирующие биологические нейроны, изображенные на рисунке ниже.

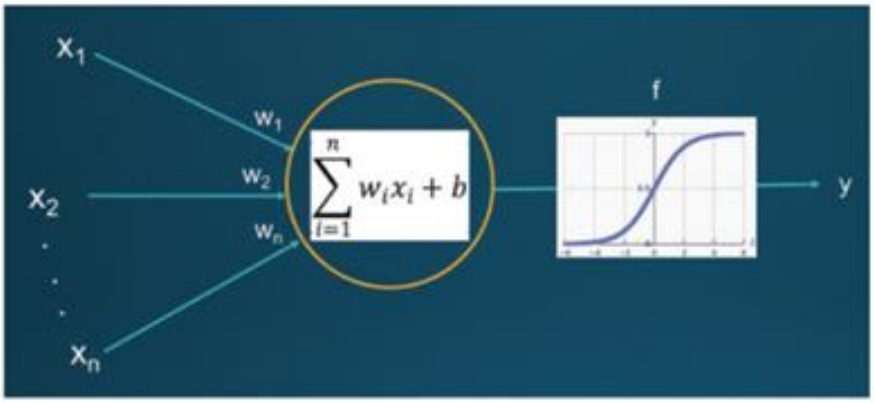


Модель биологического нейрона

Источник: https://pixabay.com/vectors/neuron-nerve-cell-axon-dendrite-296581/

Дендриты, окружающие тело клетки и показанные в левой части рисунка, являются проводниками входных сигналов от похожих клеток, а длинный аксон, исходящий из тела клетки, играет роль выхода и может соединяться с другими клетками.

Эту структуру пытается имитировать показанная ниже искусственная модель, называемая перцептроном.



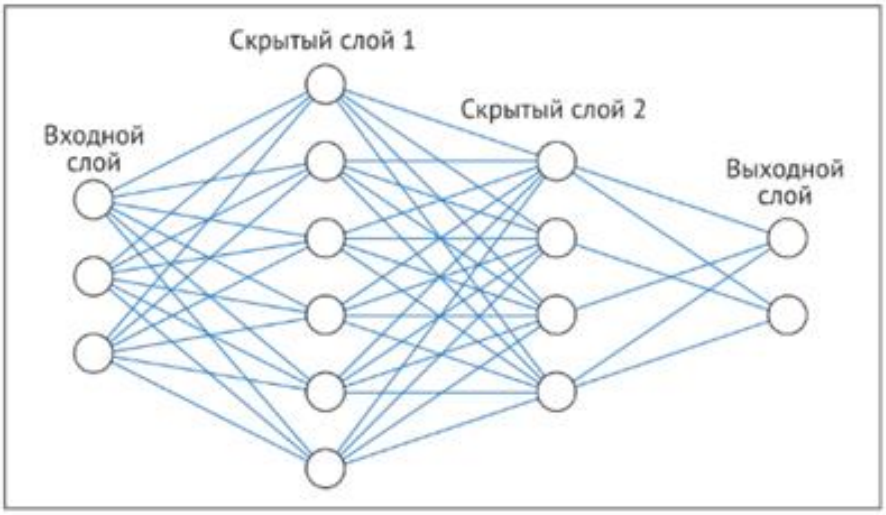
Модель искусственного нейрона – перцептрон

Перцептрон вычисляет выход, умножая каждое входное значение на некоторый вес; результаты складываются, и к сумме прибавляется смещение. Затем к результату применяется нелинейная функция активации, которая возвращает выход. Эта модель повторяет поведение биологического нейрона, который возбуждается (посылает серию импульсов через свой выходной канал), если взвешенная сумма входных сигналов оказывается больше некоторого порога.

Модель перцептрона можно использовать для простых задач классификации и регрессии, если настроить веса и смещение, так чтобы входы отображались на желаемые выходы. Но если соединить несколько перцептронов в так называемый **многослойный перцептрон**, то получается гораздо более мощная модель, которую мы опишем в следующем подразделе.

Многослойный перцептрон

Многослойный перцептрон (МСП) – это обобщение идеи перцептрона: объединение нескольких блоков, каждый из которых реализует перцептрон. Блоки в МСП собраны в слои, и каждый слой соединен со следующим. Базовая структура МСП показана на рисунке ниже.



Базовая структура многослойного перцептрона

Многослойный перцептрон состоит из трех основных частей:

- **входной слой:** получает входные значения и соединяет каждое из них с каждым нейроном следующего слоя;
- **выходной слой:** возвращает результаты, вычисленные МСП. Если МСП используется в качестве классификатора, то каждый выход представляет один класс. Если же МСП используется для регрессии, то выходной блок только один и возвращает непрерывное значение;
- **скрытые слои:** именно они составляют истинную мощь модели и придают ей сложность. На рисунке выше показано только два скрытых слоя, но их может быть и больше, разных размеров; все они располагаются между входным и выходным слоем. По мере увеличения количества скрытых слоев сеть становится глубже и может выполнять все более сложное нелинейное отображение входов на выходы.

Обучение этой модели сводится к корректировке весов и смещений каждого блока. Обычно для этого применяются алгоритмы **обратного распространения**. Их идея состоит в том, чтобы минимизировать расхождение между фактическими и желательными результатами, распространяя ошибку от выходного слоя к входному (в обратном направлении). Веса отдельных блоков – нейронов – изменяются так, что чем больше вклад нейрона в ошибку, тем сильнее корректируется его вес.

На протяжении многих лет ограничения вычислительных возможностей оборудования позволяли включать не более двух-трех скрытых слоев, но новейшие достижения полностью изменили картину. Мы объясним, что произошло, в следующем подразделе.

Глубокое обучение и сверточные нейронные сети

Сравнительно недавно алгоритмы обратного распространения совершили гигантский скачок, в результате чего стало возможно увеличить количество скрытых слоев сети. В таких глубоких нейронных сетях каждый слой может интерпретировать сравнительно простые абстрактные концепции, которым обучились предыдущие слои, и выдавать на выходе концепции более высокого уровня. Например, в задаче распознавания лиц первый слой обрабатывает пиксели изображения и обучается обнаруживать границы в разных направлениях. Следующий слой собирает границы в линии, углы и т. д., следующий за ним выделяет черты лица, например нос и губы, и, наконец, последний объединяет их и строит лицо.

Впоследствии была предложена идея **сверточной нейронной сети**, которая позволяет уменьшить количество блоков глубокой нейронной сети, обрабатывающей двумерную информацию (например, изображения), поскольку рассматривает близлежащие участки иначе, чем отдаленные. Такие модели оказываются особенно успешными при обработке изображений и видео. Помимо **полносвязных слоев**, аналогичных скрытым слоям многослойного перцептрона, в них используются **пулинговые слои** (слои **понижающей передискретизации**), которые агрегируют выходы нейронов предыдущих слоев, и **сверточные слои**, которые используются как фильтры

для обнаружения определенных признаков (например, границ определенной ориентации).

Обучение глубоких нейронных сетей требует большого объема вычислений и часто производится с применением **графических процессоров** (GPU), которые эффективнее обычных процессоров реализуют алгоритмы обратного распространения. Специализированные библиотеки глубокого обучения, например TensorFlow, умеют работать с вычислительными платформами на базе GPU. Но в этой главе мы для простоты будем пользоваться реализацией МСП, предлагаемой библиотекой `sklearn`, и простым набором данных. Однако все принципы применимы и к более сложным сетям и наборам данных.

В следующем разделе мы посмотрим, как архитектуру МСП можно оптимизировать с помощью генетического алгоритма.

Оптимизация архитектуры классификатора на основе глубокой сети

При создании модели глубокой нейронной сети для задачи машинного обучения одно из важнейших решений, которые предстоит принять, связано с выбором архитектуры сети. В случае многослойного перцептрона количество блоков во входном и выходном слоях определяется характеристиками поставленной задачи. Поэтому все решения относятся к скрытым слоям – их количеству и числу нейронов в каждом слое. Существуют эвристические соображения, упрощающие принятие таких решений, но чаще всего выбор архитектуры сводится к утомительному методу проб и ошибок.

Один из подходов к выбору параметров архитектуры сети состоит в том, чтобы рассматривать их как гиперпараметры модели, поскольку они задаются до начала обучения. В этом разделе мы примем данный подход и воспользуемся генетическим алгоритмом для поиска наилучшей комбинации скрытых слоев точно так же, как поступали при выборе лучших значений гиперпараметров в предыдущей главе. Начнем с постановки задачи – классификации ирисов.

Набор данных Iris

Один из наиболее изученных наборов данных, Iris (https://archive.ics.uci.edu/ml/datasets/Iris), содержит результаты измерений лепестков и чашелистиков трех видов ирисов (Iris setosa, Iris virginica и Iris versicolor), проделанных биологами в 1936 году.

Набор данных содержит по 50 примеров каждого из трех видов и включает следующие признаки:

- sepal_length – длина чашелистика (см);
- sepal_width – ширина чашелистика (см);
- petal_length – длина лепестка (см);
- petal_width – ширина лепестка (см).

Этот набор данных входит в состав библиотеки sklearn, для его инициализации нужно выполнить следующие действия:

```
from sklearn import datasets

data = datasets.load_iris()
X = data['data']
y = data['target']
```

В своих экспериментах мы будем использовать МСП-классификатор в сочетании с этим набором и воспользуемся возможностями генетических алгоритмов для нахождения архитектуры сети – количества скрытых слоев и числа нейронов в каждом слое, – при которой достигается наилучшая верность классификации.

Коль скоро мы работаем с генетическим алгоритмом, нужно придумать, как представить архитектуру в виде хромосомы.

Представление конфигурации скрытого слоя

Поскольку архитектура МСП определяется конфигурацией скрытого слоя, подумаем, как можно представить эту конфигурацию в нашем решении. Конфигурация скрытого слоя в модели многослойного перцептрона из библиотеки sklearn (https://scikit-learn.org/stable/modules/neural_networks_supervised.html) определяется кортежем hidden_layer_sizes, передаваемым конструктору в качестве параметра. По умолчанию этот кортеж равен (100,), т. е. имеется один скрытый слой, состоящий из 100 блоков. Чтобы сконфигурировать МСП с тремя скрытыми слоями по 20 блоков в каждом, нужно было бы задать кортеж (20, 20, 20). Прежде чем приступать к реализации оптимизатора конфигурации скрытых слоев на базе генетического алгоритма, нужно определить подходящий формат хромосомы.

Нам нужна хромосома, которая кодировала бы как количество слоев, так и число блоков в каждом слое. Один из вариантов – хромосома переменной длины, которая прямо транслируется в кортеж переменной длины, передаваемый в параметре модели hidden_layer_sizes. Однако при этом пришлось бы придумывать специальные, возможно, громоздкие генетические операторы. Чтобы можно было воспользоваться стандартными генетическими операторами, мы остановимся на представлении фиксированной длины. Тогда максимальное количество слоев задается заранее; все эти слои всегда представлены в хромосоме, но необязательно присутствуют в решении. Например, если мы решим ограничиться четырьмя скрытыми слоями, то хромосома будет выглядеть так:

$[n1, n2, n3, n4]$,

где n_i – число блоков в i-м слое.

Но для управления числом скрытых слоев сети некоторые из этих значений могут быть нулевыми или отрицательными. Это означает, что в сеть не нужно добавлять больше слоев. Проиллюстрируем на примерах.

- Хромосома [10, 20, –5, 15] транслируется в кортеж (10, 20), поскольку –5 служит признаком завершения.
- Хромосома [10, 0, –5, 15] транслируется в кортеж (10,), поскольку 0 служит признаком завершения.
- Хромосома [10, 20, 5, –15] транслируется в кортеж (10, 20, 5), поскольку –15 служит признаком завершения.
- Хромосома[10, 20, 5, 15] транслируется в кортеж (10, 20, 5, 15).

Чтобы гарантировать наличие хотя бы одного скрытого слоя, мы можем потребовать, чтобы первый параметр всегда был больше нуля. Остальные параметры могут иметь различные распределения вокруг нуля, чтобы мы могли управлять вероятностью того, что они станут признаками завершения.

Кроме того, хотя эта хромосома состоит только из целых чисел, мы решили все же работать с числами с плавающей точкой, как в предыдущей главе. Это удобно, потому что позволяет использовать существующие генетические операторы, оставив себе возможность обобщить хромосому, так чтобы она могла включать параметры других типов. Позже мы так и поступим. Числа типа float можно преобразовать в целые с помощью функции `round()`. Проиллюстрируем этот обобщенный подход на примерах.

- Хромосома [9.35, 10.71, –2.51, 17.99] транслируется в кортеж (9, 11).
- Хромосома [9.35, 10.71, 2.51, –17.99] транслируется в кортеж (9, 11, 3).

Чтобы оценить хромосому, представляющую архитектуру, мы должны преобразовать ее обратно в кортеж слоев, создать МСП-классификатор с такими слоями, обучить его и оценить верность. Как это делается, показано в следующем подразделе.

Оценка верности классификатора

Класс, инкапсулирующий оценку верности МСП-классификатора для набора данных Iris, называется `MlpLayersTest` и находится в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter09/mlp_layers_test.py.

Ниже описаны его основные функции.

1. Метод класса `convertParam()` принимает список `params`. Это хромосома, описанная в предыдущем разделе, она содержит числа с плавающей точкой, представляющие не более четырех скрытых слоев. Метод преобразует этот список в кортеж `hidden_layer_sizes`:

```
if round(params[1]) <= 0:
    hiddenLayerSizes = round(params[0]),
elif round(params[2]) <= 0:
    hiddenLayerSizes = (round(params[0]), round(params[1]))
elif round(params[3]) <= 0:
    hiddenLayerSizes = (round(params[0]), round(params[1]), round(params[2]))
else:
    hiddenLayerSizes = (round(params[0]), round(params[1]), round(params[2]),
round(params[3]))
```

2. Метод `getAccuracy()` принимает список `params`, представляющий конфигурацию скрытых слоев, с помощью метода `convertParam()` преобразует его в кортеж `hidden_layer_sizes` и инициализирует МСП-классификатор этим кортежем:

```
hiddenLayerSizes = self.convertParams(params)
self.classifier = MLPClassifier(hidden_layer_sizes=hiddenLayerSizes)
```

   Затем он определяет верность классификатора с помощью *k*-группового перекрестного контроля, как при работе с набором данных Wine:

```
cv_results = model_selection.cross_val_score(self.classifier,
                                             self.X,
                                             self.y,
                                             cv=self.kfold,
                                             scoring='accuracy')
return cv_results.mean()
```

Класс `MlpLayersTest` используется в оптимизаторе на базе генетического алгоритма, как описано в следующем подразделе.

Оптимизация архитектуры МСП с помощью генетического алгоритма

Теперь, умея представлять конфигурацию МСП для классификации данных об ирисах и зная, как определить верность классификатора для любой конфигурации, мы можем перейти к созданию генетического алгоритма для поиска оптимальной конфигурации – количества скрытых слоев (не более 4) и числа блоков в каждом слое, – при которой достигается наилучшая верность. Наше решение реализовано Python-программой, находящейся в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter09/01-optimize-mlp-layers.py.

Ниже описаны основные части этого решения.

1. Сначала зададим нижнюю и верхнюю границы чисел с плавающей точкой, представляющих скрытый слой. Для первого скрытого слоя возьмем диапазон [5, 15], а для остальных нижние границы будут монотонно уменьшающимися отрицательными значениями, так чтобы шансы слоя оказаться последним увеличивались.

```
# [layer_layer_1_size, hidden_layer_2_size,
hidden_layer_3_size, hidden_layer_4_size]
BOUNDS_LOW = [ 5, -5, -10, -20]
BOUNDS_HIGH = [15, 10, 10, 10]
```

2. Затем создадим экземпляр класса `MlpLayersTest`, с помощью которого будем проверять различные комбинации параметров архитектуры:

```
test = mlp_layers_test.MlpLayersTest(RANDOM_SEED)
```

3. Поскольку мы стремимся максимизировать верность классификатора, определим единственную цель – максимизирующую стратегию приспособления:

```
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
```

4. Теперь применим тот же подход, что в предыдущей главе: поскольку решение представлено списком чисел типа float, каждое в своем диапазоне, в цикле обойдем все пары, состоящие из нижней и верхней границ, и для каждого диапазона создадим в инструментарии свой оператор (layer_size_attribute), который впоследствии будет генерировать случайное число в этом диапазоне:

```
for i in range(NUM_OF_PARAMS):
    # "layer_size_attribute_0", "layer_size_attribute_1", ...
    toolbox.register("layer_size_attribute_" + str(i),
                     random.uniform,
                     BOUNDS_LOW[i],
                     BOUNDS_HIGH[i])
```

5. Далее создадим кортеж layer_size_attributes, содержащий только что созданные генераторы случайных чисел с плавающей точкой для каждого скрытого слоя:

```
layer_size_attributes = ()
for i in range(NUM_OF_PARAMS):
    layer_size_attributes = layer_size_attributes + \
(toolbox.__getattribute__("layer_size_attribute_" + str(i)),)
```

6. Теперь можно воспользоваться этим кортежем в сочетании со встроенным в DEAP оператором initCycle(), чтобы создать оператор individualCreator, который порождает экземпляр индивидуума со слоями случайно сгенерированных размеров:

```
toolbox.register("individualCreator",
                 tools.initCycle,
                 creator.Individual,
                 layer_size_attributes,
                 n=1)
```

7. Далее мы просим генетический алгоритм использовать метод getAccuracy() экземпляра MlpLayersTest для вычисления приспособленности. Напомним, что метод getAccuracy(), описанный в предыдущем подразделе, преобразует переданного ему индивидуума – список четырех чисел типа float – в кортеж размеров скрытых слоев. Затем этот кортеж используется для конфигурирования МСП-классификатора, после чего вычисляется его верность с помощью *k*-группового перекрестного контроля:

```
def classificationAccuracy(individual):
    return test.getAccuracy(individual),

toolbox.register("evaluate", classificationAccuracy)
```

8. Что касается генетических операторов, то мы повторяем конфигурацию, описанную в предыдущей главе. В качестве оператора отбора, как обычно, используем турнир размера 2, а для скрещивания и мутации берем операторы, специализированные для хромосом в виде списков чисел с плавающей точкой и передаем им границы, определенные нами для каждого скрытого слоя:

```
toolbox.register("select", tools.selTournament, tournsize=2)

toolbox.register("mate",
                 tools.cxSimulatedBinaryBounded,
                 low=BOUNDS_LOW,
                 up=BOUNDS_HIGH,
                 eta=CROWDING_FACTOR)

toolbox.register("mutate",
                 tools.mutPolynomialBounded,
                 low=BOUNDS_LOW,
                 up=BOUNDS_HIGH,
                 eta=CROWDING_FACTOR,
                 indpb=1.0 / NUM_OF_PARAMS)
```

9. Как и раньше, будем использовать элитистский подход, т. е. без изменения копировать лучших на данный момент индивидуумов из зала славы в следующее поколение:

```
population, logbook = elitism.eaSimpleWithElitism(population,
                                    toolbox,
                                    cxpb=P_CROSSOVER,
                                    mutpb=P_MUTATION,
                                    ngen=MAX_GENERATIONS,
                                    stats=stats,
                                    halloffame=hof,
                                    verbose=True)
```

После прогона алгоритма для 10 поколений с размером популяции 20 получается такой результат:

```
gen nevals max      avg
0   20     0.666667 0.416333
1   17     0.693333 0.487
2   15     0.76     0.537333
3   14     0.76     0.550667
4   17     0.76     0.568333
5   17     0.76     0.653667
6   14     0.76     0.589333
7   15     0.76     0.618
8   16     0.866667 0.616667
9   16     0.866667 0.666333
10  16     0.866667 0.722667
- Лучшее решение: 'hidden_layer_sizes'=(15, 5, 8), accuracy = 0.8666666666666666
```

Как видим, в рамках заданных нами ограничений лучшей оказалась комбинация трех скрытых слоев с размерами 15, 5, 8. При таких параметрах достигается верность классификации 86.7 %.

Для рассматриваемой задачи подобная верность выглядит приемлемо. Однако можно добиться большего.

Объединение оптимизации архитектуры с настройкой гиперпараметров

Занимаясь оптимизацией архитектуры сети – параметров скрытых слоев, – мы оставили для самого МСП-классификатора параметры по умолчанию. Однако в предыдущей главе мы видели, что с помощью настройки гиперпараметров можно повысить качество классификатора. Можно ли включить настройку в наш процесс оптимизации? Как вы, наверное, догадались, ответ утвердительный. Но сначала разберемся, какие именно гиперпараметры мы собираемся оптимизировать.

В реализации МСП-классификатора из библиотеки `sklearn` есть много настраиваемых гиперпараметров. Мы ограничимся следующими:

Имя	Тип	Описание	Значение по умолчанию
`activation`	`{'tanh', 'relu', 'logistic'}`	Функция активации в скрытых слоях	`'relu'`
`solver`	`{'sgd', 'adam', 'lbfgs'}`	Алгоритм оптимизации весов	`'adam'`
`alpha`	`float`	Параметр регуляризации	`0.0001`
`learning_rate`	`{'constant', 'invscaling', 'adaptive'}`	Характер изменения скорости обучения при обновлении весов	`'constant'`

В предыдущей главе мы видели, что представление хромосомы с числами типа float позволяет включать различные типы гиперпараметров в процесс оптимизации на базе генетического алгоритма. Поскольку конфигурация скрытых слоев представлена именно такой хромосомой, мы теперь можем включить в оптимизацию и другие параметры, дополнив уже имеющуюся хромосому. Ниже описано, как это сделать.

Представление решения

К уже имеющейся конфигурации сети вида [*n1, n2, n3, n4*] мы добавим еще четыре гиперпараметра:

- `activation` может принимать одно из трех значений: `'tanh'`, `'relu'` или `'logistic'`. Поэтому представим этот параметр числом типа float в диапазоне `[0, 2.99]`. Для преобразования числа типа float в одно из выше-

перечисленных значений применим к нему функцию floor(), которая возвращает значение 0, 1 или 2. Затем заменим 0 на tanh, 1 – на relu, а 2 – на logistic;

- solver может принимать одно из трех значений: 'sgd', 'adam' или 'lbfgs'. Как и параметр activation, его можно представить числом типа float в диапазоне [0, 2.99];
- alpha уже является числом с плавающей точкой, так что никакого преобразования не нужно. Оно будет ограничено диапазоном [0.0001, 2.0];
- learning_rate может принимать одно из трех значений: 'constant', 'invscaling' или 'adaptive'. И снова для его представления можно использовать число типа float в диапазоне [0, 2.99].

Вычисление верности классификатора

Оценка верности МСП-классификатора с заданной комбинацией скрытых слоев и гиперпараметров реализована в Python-классе MlpHyperparametersTest, находящемся в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter09/mlp_hyperparameters_test.py.

В основе этого класса лежит класс MlpLayersTest, которым мы пользовались, когда оптимизировали конфигурацию скрытых слоев, но с несколькими модификациями.

1. Метод convertParam() теперь обрабатывает список params, в котором первые четыре элемента (params[0], ... , params[3]), как и раньше, представляют размеры скрытых слоев, а элементы params[4], ... , params[7] – добавленные для оценки гиперпараметры. Поэтому в метод добавлены следующие строчки для преобразования дополнительных параметров в их истинные значения, которые затем можно будет подать на вход МСП-классификатора:

```
activation = ['tanh', 'relu', 'logistic'][floor(params[4])]
solver = ['sgd', 'adam', 'lbfgs'][floor(params[5])]
alpha = params[6]
learning_rate = ['constant', 'invscaling', 'adaptive'][floor(params[7])]
```

2. Аналогично метод getAccuracy() теперь обрабатывает пополненный список params. Он конфигурирует МСП-классификатор преобразованными значениями всех параметров, а не только задает конфигурацию скрытых слоев.

```
hiddenLayerSizes, activation, solver, alpha, learning_rate =
self.convertParams(params)

self.classifier = MLPClassifier(random_state=self.randomSeed,
                                 hidden_layer_sizes=hiddenLayerSizes,
                                 activation=activation,
                                 solver=solver,
                                 alpha=alpha,
                                 learning_rate=learning_rate)
```

Этот класс `MlpHyperparametersTest` используется в генетическом оптимизаторе, как описано в следующем подразделе.

Оптимизация объединенной конфигурации МСП с помощью генетического алгоритма

Генетический поиск лучшей комбинации скрытых слоев и гиперпараметров реализован в программе `02-optimize-mlphyperparameters.py`, которая находится в файле по адресу https://github.com/PacktPublishing/Hands-On-Genetic-Algorithms-with-Python/blob/master/Chapter09/02-optimize-mlphyperparameters.py.

Благодаря унифицированному представлению всех параметров числами с плавающей точкой эта программа почти идентична программе из предыдущего раздела. Главное отличие – определение списков `BOUNDS_LOW` и `BOUNDS_HIGH`, которые содержат диапазоны параметров. К четырем прежним диапазонам – по одному для каждого скрытого слоя – мы добавили еще четыре, соответствующих четырем рассмотренным выше гиперпараметрам:

```
# 'hidden_layer_sizes': первые четыре значения
# 'activation': 0..2.99
# 'solver': 0..2.99
# 'alpha': 0.0001..2.0
# 'learning_rate': 0..2.99
BOUNDS_LOW = [ 5, -5, -10, -20, 0, 0, 0.0001, 0 ]
BOUNDS_HIGH = [15, 10, 10, 10, 2.999, 2.999, 2.0, 2.999]
```

И это все – теперь программа может обработать дополнительные параметры без каких-либо новых изменений.

После прогона алгоритма для 5 поколений с размером популяции 20 получается такой результат:

```
gen nevals max      avg
0   20     0.933333 0.447333
1   16     0.933333 0.631667
2   15     0.94     0.736667
3   16     0.94     0.849
4   15     0.94     0.889667
5   17     0.946667 0.937
- Лучшее решение:
'hidden_layer_sizes'=(8, 8)
'activation'='relu'
'solver'='lbfgs'
'alpha'=0.572971105096338
'learning_rate'='invscaling'
=> верность = 0.9466666666666667
```



Из-за различий между операционными системами результаты в вашей системе могут немного отличаться от показанных выше.

Отсюда следует, что в пределах определенных нами диапазонов лучшей комбинацией скрытых слоев и гиперпараметров является следующая:

- два скрытых слоя по 8 блоков в каждом;
- параметр `activation` равен `'relu'` – это значение по умолчанию;
- параметр `solver` равен `'lbfgs'`, а не `'adam'`, как по умолчанию;
- параметр `learning_rate` равен `'invscaling'`, а не `'constant'`, как по умолчанию;
- параметр `alpha` равен 0.572 – значительно больше, чем значение по умолчанию 0.0001.

При такой комбинированной оптимизации точность классификации равна приблизительно 94.7 % – существенное улучшение предыдущих результатов, достигнутое при меньшем числе скрытых слоев и блоков в каждом слое.

Резюме

В этой лабораторной работе мы познакомились с основными понятиями искусственных нейронных сетей и глубокого обучения. Мы рассмотрели набор данных Iris и классификатор на основе **многослойного перцептрона (МСП),** а затем определили, что понимается под оптимизацией архитектуры сети. Далее продемонстрировали оптимизацию архитектуры МСП-классификатора с помощью генетического алгоритма. Наконец, мы объединили оптимизацию архитектуры сети и гиперпараметров модели в одном генетическом алгоритме и сумели еще улучшить качество классификатора.

Задание для самостоятельной работы.

1. Изучите библиотеку sklear (Scikit-Lean) и Scikit-Learn datasets

2. Изучить раздел «Оптимизация архитектуры МСП с помощью генетического алгоритма». Изучить влияние параметров ГА (число поколений, размер популяции, скрещивание, мутация) на параметры архитектуры МСП, оценить верность классификации.

3. Изучить раздел «Объединение оптимизации архитектуры с настройкой гиперпараметров».

4. Использовать произвольный датасет (Scikit-Learn datasets) и выполнить:

- «Оптимизация архитектуры МСП с помощью генетического алгоритма»
- «Объединение оптимизации архитектуры с настройкой гиперпараметров».